

Dedication

To my dear parents,

*for their unconditional love, sacrifices, prayers, and
constant support.*

To my family,

*for their encouragement, patience, and confidence
throughout this journey.*

To my teachers and mentors,

for their guidance, inspiration, and valuable advice.

I dedicate this work with deep gratitude and respect.

AIT-LAHCEN REDOUAN

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor for the guidance, availability, constructive feedback, and encouragement provided throughout this final year project. The advice received helped me clarify the project scope, strengthen the technical choices, and maintain a rigorous engineering approach.

I would also like to thank the academic staff for the knowledge and methodological foundations shared during my Master's program. Their teaching contributed directly to my understanding of data engineering, distributed systems, databases, software development, and reliable information system design.

My appreciation goes to the professional environment that supported the context of this project and helped me understand practical data engineering requirements. Working under the broader AdOptimizer AI initiative allowed me to position this work as a distributed customer data lakehouse platform focused on ingestion, distributed storage, lakehouse organization, transformation, quality validation, orchestration, observability, and security hardening.

I am grateful to my colleagues and classmates for their discussions, shared ideas, and support during this academic journey. I also extend my deepest thanks to my family for their patience, encouragement, and confidence during the long periods of research, implementation, testing, and documentation.

Finally, I would like to thank the members of the jury for the time and attention devoted to evaluating this work. I hope this report clearly reflects the effort invested in designing and implementing a modern, distributed, secure, and production-oriented data engineering platform.

Abstract

Modern organizations increasingly rely on data produced from multiple heterogeneous sources, including transactional systems, e-commerce platforms, behavioral event streams, and customer profile datasets. However, transforming these raw inputs into reliable, governed, and analytics-ready data products requires a structured data engineering platform capable of ingestion, storage, distributed processing, validation, orchestration, and monitoring.

This final year project presents the design and implementation of AdOptimizer CDP, a distributed customer data lakehouse platform developed under the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES. The platform focuses on building a scalable data engineering foundation for multi-source customer data. It integrates Apache Kafka for ingestion, Hadoop HDFS for distributed storage, Apache Spark for distributed processing, Apache Iceberg and Hive Metastore for lakehouse table management, Trino for SQL exploration, dbt-spark for transformation modeling, Great Expectations for data quality validation, Apache Airflow for orchestration, and Prometheus with Grafana for observability.

The project demonstrates an end-to-end architecture that transforms raw customer-related datasets into structured lakehouse layers and trusted analytical data products. The implemented solution emphasizes modularity, reproducibility, scalability, data quality, observability, security, and readiness for future analytics and machine learning use cases.

Keywords : Big Data, Data Engineering, Distributed Systems, Data Lakehouse, Apache Kafka, HDFS, Apache Spark, Apache Iceberg, Trino, dbt-spark, Data Quality, Observability, Security.

Résumé

Les organisations modernes exploitent des données issues de sources hétérogènes telles que les systèmes transactionnels, les plateformes e-commerce, les événements comportementaux et les données de profil client. Cependant, la transformation de ces données brutes en produits de données fiables, gouvernés et exploitables nécessite une plateforme d'ingénierie de données structurée, capable d'assurer l'ingestion, le stockage, le traitement distribué, la validation, l'orchestration et la supervision.

Ce projet de fin d'études présente la conception et la mise en oeuvre de AdOptimizer CDP, une plateforme lakehouse de données client distribuée réalisée dans le cadre de l'initiative plus large AdOptimizer AI chez SMART AUTOMATION TECHNOLOGIES. Le projet se concentre sur la construction d'une fondation d'ingénierie de données scalable pour des données client multi-sources. L'architecture intègre Apache Kafka pour l'ingestion, Hadoop HDFS pour le stockage distribué, Apache Spark pour le traitement distribué, Apache Iceberg et Hive Metastore pour la gestion des tables lakehouse, Trino pour l'exploration SQL, dbt-spark pour les transformations, Great Expectations pour la qualité des données, Apache Airflow pour l'orchestration, ainsi que Prometheus et Grafana pour l'observabilité.

La solution réalisée illustre une chaîne de traitement complète permettant de convertir des données client brutes en couches lakehouse structurées et en produits de données analytiques fiables. Elle met l'accent sur la modularité, la reproductibilité, la scalabilité, la qualité des données, l'observabilité, la sécurité et la préparation de futurs cas d'usage analytiques et machine learning.

Mots-clés : Big Data, Ingénierie des données, Systèmes distribués, Data Lakehouse, Apache Kafka, HDFS, Apache Spark, Apache Iceberg, Trino, dbt-spark, Qualité des données, Observabilité, Sécurité.

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
Résumé	iv
List of Acronyms and Abbreviations	xi
1 General Project Context	1
1.1 Introduction	1
1.2 Company and Strategic Context	1
1.3 Problem Statement	1
1.4 Project Positioning	2
1.5 Chapter Conclusion	2
2 State of the Art and Theoretical Background	3
2.1 Introduction	3
2.2 Customer Data Platforms and Customer 360	3
2.3 Data Lakes, Warehouses, and Lakehouses	3
2.4 Distributed Data Systems	3
2.5 Workflow Orchestration and Data Quality	4
2.6 Security in Distributed Data Platforms	4
2.7 Chapter Conclusion	4

3	Technology Watch	5
3.1	Introduction	5
3.2	Selection Criteria	5
3.3	Ingestion and Messaging Technologies	5
3.4	Storage and Lakehouse Technologies	5
3.5	Processing, Query, and Transformation Technologies	6
3.6	Quality, Orchestration, and Monitoring Technologies	6
3.7	Security Technology Choices	6
3.8	Chapter Conclusion	6
4	Requirements Analysis and System Design	7
4.1	Introduction	7
4.2	Functional Requirements	7
4.3	Non-Functional Requirements	7
4.4	Actors and Usage Perspective	7
4.5	Global Logical Architecture	8
4.6	Physical Multi-Node Architecture	8
4.7	Design Rationale	8
4.8	Chapter Conclusion	8
5	Implementation of the Distributed Lakehouse Platform	9
5.1	Introduction	9
5.2	Data Sources and Dataset Modeling	9
5.3	Kafka-Based Ingestion Layer	9
5.4	HDFS Bronze Storage Layer	9
5.5	Spark Processing and Iceberg Loading	10
5.6	Hive Metastore and Catalog Governance	10
5.7	Trino Query Layer	10
5.8	dbt-spark Transformation Layer	10
5.9	Great Expectations Validation Layer	10

5.10	Airflow Orchestration Layer	10
5.11	Chapter Conclusion	10
6	Deployment Strategy and Distributed Execution	11
6.1	Introduction	11
6.2	Control Plane Responsibilities	11
6.3	VM-Level Service Separation	11
6.4	Data Flow Across Machines	11
6.5	Distributed Processing Proof	12
6.6	Operational Packaging and Deployment Bundles	12
6.7	Chapter Conclusion	12
7	Security Architecture and Hardening	13
7.1	Introduction	13
7.2	Security Objectives	13
7.3	Kerberos-Secured Hadoop Services	13
7.4	SPNEGO-Protected HDFS Browser Access	13
7.5	TLS-Protected Interfaces	14
7.6	Authenticated Trino Access	14
7.7	SSH-Based Remote Spark Submission	14
7.8	Encrypted Runtime Storage Support	14
7.9	Security Boundaries and Remaining Limits	14
7.10	Chapter Conclusion	14
8	Validation, Testing, and Results	15
8.1	Introduction	15
8.2	Validation Strategy	15
8.3	Infrastructure Validation	15
8.4	Pipeline Validation	15
8.5	Distributed Execution Validation	16
8.6	Security Validation	16

8.7	Observability and Monitoring Validation	16
8.8	Main Results	16
8.9	Chapter Conclusion	16
9	Discussion and Critical Analysis	17
9.1	Introduction	17
9.2	Strengths of the Final Architecture	17
9.3	Trade-Offs and Constraints	17
9.4	Academic Value	17
9.5	Industrial Value	18
9.6	Critical Limits	18
9.7	Chapter Conclusion	18
10	General Conclusion and Perspectives	19
10.1	General Conclusion	19
10.2	Main Achievements	19
10.3	Future Perspectives	19
10.4	Final Positioning	20
A	Appendices	22
A.1	Appendix Organization	22
A.2	Recommended Appendix Contents	22
A.3	Evidence Location	22

List of Figures

List of Tables

List of Acronyms and Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CDP	Customer Data Platform
DAG	Directed Acyclic Graph
dbt	Data Build Tool
ETL	Extract, Transform, Load
GE	Great Expectations
HDFS	Hadoop Distributed File System
JDBC	Java Database Connectivity
JSONL	JSON Lines
KDC	Key Distribution Center
KPI	Key Performance Indicator
LTV	Lifetime Value
ML	Machine Learning
PFE	Projet de Fin d'Etudes
RBAC	Role-Based Access Control
SQL	Structured Query Language
SPNEGO	Simple and Protected GSSAPI Negotiation Mechanism
TLS	Transport Layer Security
UI	User Interface
VM	Virtual Machine

Chapter 1

General Project Context

1.1 Introduction

This chapter introduces the academic and industrial context of AdOptimizer CDP. It clarifies the company environment, the broader strategic initiative, the motivation for the project, and the positioning of the PFE work within the larger transformation objective.

1.2 Company and Strategic Context

AdOptimizer CDP is developed within AdOptimizer AI at SMART AUTOMATION TECHNOLOGIES. The global initiative aims to strengthen data-driven marketing and customer intelligence capabilities. Within this larger vision, the PFE contribution focuses specifically on the distributed and secure data platform required to centralize, process, and expose customer data reliably.

1.3 Problem Statement

Traditional customer data environments are often fragmented across multiple channels, technologies, and business silos. This fragmentation reduces data consistency, limits analytical agility, and makes future intelligent services more difficult to implement. A distributed lakehouse platform addresses these issues by combining scalable storage, structured governance, analytical query access, and orchestrated processing.

1.4 Project Positioning

The project must be defended as a distributed customer data engineering platform rather than as a standalone dashboard, a simple ETL chain, or a finished AI model. Its purpose is to provide the infrastructure and governance foundation upon which future Customer 360 and machine learning services can be built.

1.5 Chapter Conclusion

This chapter establishes the academic and business framing of the project and prepares the transition toward the theoretical foundations presented in the next chapter.

Chapter 2

State of the Art and Theoretical Background

2.1 Introduction

This chapter develops the academic foundation required to defend the project rigorously. Its purpose is to establish the major concepts, architectural trade-offs, and distributed data system principles that justify the final design choices adopted in AdOptimizer CDP.

2.2 Foundations of Modern Distributed Data Systems

This section should explain how data systems evolved from centralized architectures toward distributed platforms capable of handling scale, heterogeneity, and operational complexity. It should define the importance of horizontal scaling, service distribution, and the system qualities of reliability, scalability, and maintainability. It should also introduce the notion of data-intensive applications and explain why these concerns are central to the academic positioning of AdOptimizer CDP.

2.3 Core Theoretical Trade-offs in Distributed Data Engineering

This section should present the essential trade-offs that shape distributed data system design. It should cover the CAP theorem, consistency trade-offs, ACID and BASE models, data partitioning, replication, fault tolerance, service resilience, and the difference between batch and stream processing. It should also explain schema-on-write and schema-on-read

as major design choices that influence storage and analytical flexibility.

2.4 Big Data Foundations

This section should establish the broader Big Data context of the project. It should explain the Five Vs of Big Data, discuss heterogeneous customer-oriented data sources, and present the core principles of distributed storage and distributed computation that support modern data platforms.

2.5 Evolution of Analytical Data Architectures

This section should compare the major architectural paradigms that emerged in response to distributed data challenges. It should cover traditional data warehouses, data lakes, data lakehouses, Lambda architecture, and Kappa architecture. The section should end with a comparative analysis showing why the lakehouse model best fits the goals of AdOptimizer CDP.

2.6 Foundational Concepts for Governance and Trust in Data Platforms

This section should explain the theoretical role of metadata, cataloging, data discovery, lineage, traceability, and data quality dimensions. It should also introduce observability as a principle for understanding, controlling, and validating distributed data operations.

2.7 Security Foundations for Distributed Data Platforms

This section should present the theoretical basis for authentication, authorization, encryption in transit, encryption at rest, and secure service exposure in multi-node environments. The objective is to prepare the reader for the security controls implemented later in the report.

2.8 Positioning of the Present Work

This section should connect the theoretical foundations of the chapter to the present project. It should explain how AdOptimizer CDP aligns with distributed lakehouse principles, why it must be defended as a data engineering platform rather than a final AI product, and how it is positioned within the broader AdOptimizer AI initiative.

2.9 Chapter Conclusion

This chapter provides the theoretical background that justifies the technical design and implementation choices documented in the following chapters. It must create a strong academic transition toward the requirements analysis and system design phase of the report.

Chapter 3

Technology Watch

3.1 Introduction

This chapter explains the technology selection process used to assemble the final platform.

3.2 Selection Criteria

The comparison should consider compatibility with distributed deployment, maturity, documentation quality, ecosystem adoption, security capabilities, operational simplicity, and suitability for academic demonstration.

3.3 Ingestion and Messaging Technologies

This section should explain the selection of Kafka for event-driven ingestion and controlled topic-based data movement.

3.4 Storage and Lakehouse Technologies

This section should justify the use of HDFS for distributed storage and Iceberg for managed analytical table layers.

3.5 Processing, Query, and Transformation Technologies

This section should justify the use of Spark for distributed processing, Trino for SQL exposure, and dbt-spark for transformation governance.

3.6 Quality, Orchestration, and Monitoring Technologies

This section should justify Great Expectations, Airflow, Prometheus, Grafana, and supporting observability services.

3.7 Security Technology Choices

This section should justify Kerberos, SPNEGO, TLS, SSH-based submission, and encrypted storage support.

3.8 Chapter Conclusion

This chapter should show that the final stack was selected for coherence, defensibility, and practical integration rather than for tool accumulation.

Chapter 4

Requirements Analysis and System Design

4.1 Introduction

This chapter formalizes the functional and non-functional requirements of AdOptimizer CDP and translates them into an architectural design.

4.2 Functional Requirements

The platform must support heterogeneous data ingestion, distributed storage, distributed transformation, catalog integration, SQL access, orchestration, monitoring, and validation workflows.

4.3 Non-Functional Requirements

The platform must demonstrate scalability, modularity, security, observability, service separation, reproducibility, and suitability for academic demonstration.

4.4 Actors and Usage Perspective

This section should identify key actors such as data engineers, analysts, platform administrators, and future data science consumers.

4.5 Global Logical Architecture

This section should present the logical data flow from ingestion to validation and analytical exposure.

4.6 Physical Multi-Node Architecture

This section should present the role separation across the control plane, VM1, VM2, and VM3.

4.7 Design Rationale

The final design should be explained as a balance between academic rigor, distributed execution proof, security hardening, and resource constraints.

4.8 Chapter Conclusion

This chapter provides the design baseline for the concrete implementation detailed in the next chapter.

Chapter 5

Implementation of the Distributed Lakehouse Platform

5.1 Introduction

This chapter presents the concrete implementation of AdOptimizer CDP from ingestion to analytical serving.

5.2 Data Sources and Dataset Modeling

This section should describe the selected customer-oriented datasets, the reason for their retention, and their role in the platform.

5.3 Kafka-Based Ingestion Layer

This section should describe topic organization, producers, consumers, and ingestion orchestration logic.

5.4 HDFS Bronze Storage Layer

This section should describe bronze storage organization, batch landing patterns, and the role of distributed persistence.

5.5 Spark Processing and Iceberg Loading

This section should explain Spark jobs, schema handling, loading behavior, and the creation of analytical Iceberg tables.

5.6 Hive Metastore and Catalog Governance

This section should explain how metadata is centralized and exposed to upper analytical services.

5.7 Trino Query Layer

This section should explain SQL access patterns, query federation role, and secure access exposure.

5.8 dbt-spark Transformation Layer

This section should explain transformation modeling, model organization, and governed analytical layer creation.

5.9 Great Expectations Validation Layer

This section should explain quality checks and their role in proving governance and trustworthiness.

5.10 Airflow Orchestration Layer

This section should explain the DAG structure, sequencing, operational controls, and restart logic.

5.11 Chapter Conclusion

This chapter should demonstrate that the platform is implemented as a coherent system rather than as isolated components.

Chapter 6

Deployment Strategy and Distributed Execution

6.1 Introduction

This chapter explains how the platform is deployed physically and how distributed execution is demonstrated across multiple nodes.

6.2 Control Plane Responsibilities

This section should describe the local control plane services, their role, and why they remain separated from the data-plane virtual machines.

6.3 VM-Level Service Separation

This section should explain why VM1, VM2, and VM3 host different services and how this separation supports resilience, modularity, and clearer operational reasoning.

6.4 Data Flow Across Machines

The report should clearly explain the end-to-end movement of data:

- source datasets to Kafka,
- Kafka to HDFS bronze,

- HDFS to Spark,
- Spark to Iceberg,
- Iceberg and Metastore to Trino,
- transformations and validation to final analytical exposure.

6.5 Distributed Processing Proof

This section should explain how Spark workers collaborate across nodes and how the deployment proves multi-node execution rather than single-machine processing.

6.6 Operational Packaging and Deployment Bundles

This section should document the deployment bundles, environment preparation, startup scripts, and operational checks used to launch the platform.

6.7 Chapter Conclusion

This chapter should prove that the deployment is intentionally distributed, reproducible, and academically defensible.

Chapter 7

Security Architecture and Hardening

7.1 Introduction

This chapter explains the security controls introduced into AdOptimizer CDP and their role in protecting the distributed platform.

7.2 Security Objectives

The security layer aims to protect service access, secure browser-facing interfaces, control distributed identity, and reduce exposure of operational services.

7.3 Kerberos-Secured Hadoop Services

This section should explain the use of Kerberos for Hadoop authentication and why secure identity exchange is important in a distributed system.

7.4 SPNEGO-Protected HDFS Browser Access

This section should explain how browser access to HDFS was protected and why this matters for administrative interfaces.

7.5 TLS-Protected Interfaces

This section should explain the TLS layer added to exposed services and the distinction between internal service traffic and browser-facing endpoints.

7.6 Authenticated Trino Access

This section should explain Trino gateway protection, authenticated access, and the user-facing implications of secure SQL exposure.

7.7 SSH-Based Remote Spark Submission

This section should explain how remote execution from the control plane to VM2 is secured operationally.

7.8 Encrypted Runtime Storage Support

This section should explain the role of encrypted runtime storage and its contribution to protecting data at rest.

7.9 Security Boundaries and Remaining Limits

This section should present the implemented scope honestly and distinguish completed hardening from future enterprise-grade enhancements.

7.10 Chapter Conclusion

This chapter should show that security was integrated as an architectural concern rather than added as a superficial afterthought.

Chapter 8

Validation, Testing, and Results

8.1 Introduction

This chapter presents the validation methodology and the main technical results obtained from the deployed platform.

8.2 Validation Strategy

The validation should cover service readiness, distributed execution, ingestion success, storage correctness, analytical exposure, transformation success, quality controls, and observability.

8.3 Infrastructure Validation

This section should summarize the checks performed for Kafka, HDFS, Hive Metastore, Spark, Trino, Kerberos, TLS gateways, and monitoring services.

8.4 Pipeline Validation

This section should present the main DAG execution results and explain how the orchestration proves operational readiness.

8.5 Distributed Execution Validation

This section should provide evidence that multiple Spark workers and virtual machines collaborate during processing.

8.6 Security Validation

This section should present authentication and protected-access checks, especially for HDFS browser access and secured analytical services.

8.7 Observability and Monitoring Validation

This section should summarize the role of Grafana, Prometheus, cAdvisor, and pipeline metrics in validating operational visibility.

8.8 Main Results

This section should present the overall technical outcome: the platform is deployed, secure, distributed, orchestrated, and usable as a foundation for future higher-level analytics.

8.9 Chapter Conclusion

This chapter should demonstrate that the project is operationally validated rather than only theoretically designed.

Chapter 9

Discussion and Critical Analysis

9.1 Introduction

This chapter provides a critical interpretation of the implemented work.

9.2 Strengths of the Final Architecture

This section should analyze the main strengths of the project, including modularity, service separation, distributed execution, governance orientation, and security integration.

9.3 Trade-Offs and Constraints

This section should discuss practical constraints such as limited infrastructure resources, multi-service integration complexity, and the operational overhead of a secure distributed architecture.

9.4 Academic Value

This section should explain the academic contribution of the project in terms of data engineering, distributed systems, lakehouse design, orchestration, observability, and platform security.

9.5 Industrial Value

This section should explain how the platform creates a usable foundation for future customer intelligence and AI services inside the broader initiative.

9.6 Critical Limits

This section should clearly distinguish what was intentionally implemented from what remains outside the current PFE scope.

9.7 Chapter Conclusion

This chapter should show maturity, self-criticism, and a realistic understanding of the delivered platform.

Chapter 10

General Conclusion and Perspectives

10.1 General Conclusion

AdOptimizer CDP delivers a distributed and secured customer data lakehouse platform implemented within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES. The project demonstrates that customer-oriented data can be ingested, stored, processed, transformed, validated, exposed, orchestrated, observed, and protected within a coherent multi-node architecture.

10.2 Main Achievements

The project establishes a practical foundation that combines:

- distributed ingestion and storage,
- distributed processing,
- governed analytical exposure,
- orchestration,
- observability,
- and integrated security controls.

10.3 Future Perspectives

The platform can later support:

- Customer 360 consolidation,
- customer segmentation,
- churn prediction,
- lifetime value estimation,
- personalization services,
- and AI-driven marketing decision support.

10.4 Final Positioning

The final outcome should be defended as a strong data engineering and distributed systems contribution that prepares the path for future intelligent services without claiming to already be the final AI product.

Bibliography

Appendix A

Appendices

A.1 Appendix Organization

This appendix section is prepared to host the supporting material required for the final submission.

A.2 Recommended Appendix Contents

The final version can include:

- deployment commands,
- environment variables,
- configuration extracts,
- screenshots used as validation evidence,
- service access matrices,
- and operational troubleshooting notes.

A.3 Evidence Location

The corresponding raw support material should be organized primarily inside:

- 90_assets/
- 91_evidence/

- 99_submission_package/